

Integrantes: Ezequiel Daher; Alex Claros; Bautista Quesada.
 Materia: Programación Orientada a Objetos.
 Carrera: Tec. Desarrollo de Software.

Jerarquía de Excepciones Personalizadas

Se creó el paquete exception/ con una jerarquía encabezada por ClinicaException. Se eligió extender RuntimeException para mantener las firmas de los métodos limpias (las capas intermedias no necesitan declarar throws en cada método) y centralizar el manejo de errores en el MenuController con bloques try-catch acotados a cada caso de uso. Cada excepción expone un único constructor que recibe el mensaje descriptivo a mostrar al usuario.

Excepción	Cuando se lanza
ClinicaException	Clase base de la jerarquía. Extiende RuntimeException y no se lanza directamente: las demás excepciones del proyecto heredan de ella, lo que permite capturarlas todas con un único catch en la capa de UI.
PacienteNoEncontradoException	Se lanza al buscar un paciente por ID o DNI que no existe en el repositorio (PacienteServiceImpl y TurnoServiceImpl al validar el paciente de un turno).
OdontologoNoEncontradoException	Se lanza al buscar un odontólogo por ID o matrícula que no existe (OdontologoServiceImpl y TurnoServiceImpl al validar el odontólogo de un turno).
TurnoYaReservadoException	Se lanza al intentar registrar o modificar un turno cuando el odontólogo ya tiene otro turno asignado en la misma fecha y hora.
DatoInvalidoException	Excepción genérica para violaciones de validación: campos vacíos, IDs o DNIs no positivos, DNI o matrícula duplicados, motivo de consulta vacío, rango de fechas invertido, entidad no encontrada en casos donde no se requiere una excepción más específica, etc.

La estrategia adoptada separa la detección del error (capas service/repository) de su presentación al usuario (capa view). Los servicios validan, detectan condiciones inválidas y lanzan la excepción correspondiente. Los controllers de dominio (PacienteController, OdontologoController, etc.) simplemente propagan las excepciones, ya que no tienen contexto para presentarlas. Finalmente, el MenuController envuelve cada operación del menú en un bloque try-catch que captura ClinicaException (mostrando el mensaje al usuario vía VistaMenu.mostrarError) y, como red de seguridad, cualquier otra RuntimeException inesperada.

Integrantes: Ezequiel Daher; Alex Claros; Bautista Quesada.

Materia: Programación Orientada a Objetos.

Carrera: Tec. Desarrollo de Software.

Beneficios concretos del refactor:

- Se eliminaron los retornos null y las impresiones de error desde los servicios, evitando NullPointerException en quienes consumen sus resultados.
- Se validan IDs y DNIs antes de acceder a las colecciones, evitando accesos espurios.
- El usuario recibe un único formato de error: el mensaje descriptivo precedido por un prefijo visual (!!).
- El bucle del menú nunca se interrumpe por un error: el catch absorbe la excepción, muestra el mensaje y vuelve al submenú para que el usuario reintente.
- Las operaciones críticas como registrar un turno se vuelven transaccionales en intención: si cualquier validación falla, no se guarda nada en el repositorio.

Uso Avanzado de Colecciones

Se aprovechó la Stream API y los contratos Comparable/Comparator para implementar las consultas pedidas y reescribir filtros que antes usaban bucles explícitos. Las principales incorporaciones son:

- Paciente implementa Comparable<Paciente> ordenando por apellido y, en caso de empate, por nombre.
- PacienteServiceImpl.listarOrdenadosPorApellido() utiliza stream().sorted().collect(...), apoyándose en el compareTo.
- TurnoServiceImpl.POR_FECHA_HORA expone un Comparator<Turno> reutilizable, usado por todas las consultas de turnos para devolverlas cronológicamente ordenadas.
- TurnoServiceImpl.buscarPorRangoFechas(desde, hasta) filtra con stream().filter(...).sorted(...).collect(...).
- TurnoServiceImpl.buscarPorOdontólogoYPaciente(idO, idP) encadena dos filter para combinar criterios.
- Las consultas listarPorPaciente / listarPorOdontólogo / listarPorSecretaria fueron reescritas con streams, manteniendo la firma original.

Diagramas de Secuencia

Caso de uso: Alta de un nuevo Turno

Este diagrama representa el flujo principal del alta de un turno desde que la Secretaria ingresa los datos por consola hasta que se persiste en el TurnoRepository. Las interacciones se ordenan en una línea de tiempo descendente entre las lifelines de los objetos VistaTurno, MenuController, TurnoController, TurnoServiceImpl, PacienteRepository, OdontologoRepository y TurnoRepository. Las barras de activación verticales sobre cada lifeline indican el período en que cada objeto está ejecutando código como parte de la operación.

Pasos principales:

- VistaTurno.pedirDatosTurno() captura DNIs, fecha, hora y motivo de consulta.
- MenuController invoca TurnoController.registrarTurno(...), que delega en TurnoServiceImpl.registrarTurno(...).
- El servicio verifica la existencia del paciente (PacienteRepository.buscarPorId) y del odontólogo (OdontologoRepository.buscarPorId).

Integrantes: Ezequiel Daher; Alex Claros; Bautista Quesada.

Materia: Programación Orientada a Objetos.

Carrera: Tec. Desarrollo de Software.

- Se ejecuta la validación interna de fecha, hora y motivo (representada como llamada reflexiva validarDatos() sobre la propia lifeline del servicio).
- Se consulta TurnoRepository.existeConflictoHorario(...) para verificar disponibilidad.
- Si todo es válido, se invoca TurnoRepository.guardar(turno) y se retorna el Turno creado por la cadena de llamadas hasta la vista, que lo muestra con mostrarTurno(turno).

Caso de uso: Búsqueda de Paciente por DNI

Modela el flujo principal (camino exitoso) de la búsqueda de un paciente a partir de su DNI.

Lifelines participantes: VistaPaciente, MenuController, PacienteController, PacienteServiceImpl y PacienteRepository. Las barras de activación muestran cómo la llamada se anida desde la vista hasta el repositorio y la respuesta vuelve por el mismo camino.

Pasos principales:

- VistaPaciente solicita el DNI al usuario.
- La petición se propaga: MenuController → PacienteController.buscarPacientePorDni(dni) → PacienteServiceImpl.buscarPorDni(dni).
- El servicio realiza una validación interna del DNI (llamada reflexiva validarDni(dni) sobre la propia lifeline).
- Luego invoca PacienteRepository.buscarPorDni(dni) para recuperar el paciente.
- El Paciente retornado sube por la cadena de llamadas hasta VistaPaciente, que lo presenta mediante mostrarPaciente(p).